

AWS Certified AI Practitioner

Domain 3.1

Design Considerations for Applications That Use Foundation Models

Study Notes + Keyword Memorization

Scope note

Domain 3 represents 28% of the scored exam content. This PDF covers Task Statement 3.1 only. Official exam objectives are preserved; additional explanations are labeled as study explanations.

Prepared for revision and handwritten memorization

1. Official Task 3.1 Scope

The AWS Certified AI Practitioner Exam Guide v1.1 lists six objectives for Task Statement 3.1:

- Choose foundation models using criteria such as cost, modality, latency, multilingual capability, model size, model complexity, customization, input/output length, and prompt caching.
- Describe how inference parameters, including temperature and input/output length, affect model responses.
- Define Retrieval Augmented Generation (RAG) and describe business applications such as Amazon Bedrock Knowledge Bases.
- Identify AWS services that help store embeddings within vector databases, including Amazon OpenSearch Service, Amazon Aurora, Amazon Neptune, and Amazon RDS for PostgreSQL.
- Explain cost tradeoffs among FM customization approaches: pre-training, fine-tuning, in-context learning, RAG, and model distillation.
- Define the role of AI agents and describe business applications of agents.

Exam mindset

AIF-C01 is a foundational practitioner exam. Focus on identifying the correct concept or AWS service from a business scenario, not on implementing database schemas or coding training pipelines.

2. Choosing a Foundation Model

Criterion	What it means	Exam keyword / clue
Cost	How much the model/application costs to run.	cost-sensitive, reduce cost
Modality	What input/output types the model supports.	image + text, multimodal
Latency	How quickly a response is returned.	real-time, interactive, low latency
Multilingual	Ability to understand/generate multiple languages.	many languages, global users
Model size	Smaller vs larger model footprint/capability.	smaller, faster, cheaper vs larger capability
Model complexity	How sophisticated a model is needed for the task.	simple classification vs complex reasoning
Customization	Whether standard model behavior/knowledge is enough.	customize, domain-specific
Input/output length	How much content can go in/out.	long document, context window, response length
Prompt caching	Reuse repeated prompt context to avoid repeatedly processing the same content.	repeated system prompt, repeated context

High-yield rule

The biggest model is not automatically the best model. Choose a model that satisfies the business requirements for capability, latency, cost, modality, and context length.

3. Inference Parameters

3.1 Temperature

Setting	Typical effect	Good fit
Lower temperature	More predictable, consistent, less random	Factual Q&A, repeatable responses

Higher temperature	More diverse, creative, less deterministic	Brainstorming, creative generation
--------------------	--	------------------------------------

Keyword memory

LOW temperature = predictable / consistent. HIGH temperature = creative / diverse.

3.2 Input and output length

- Longer input means more tokens must be processed.
- Longer output means more tokens are generated.
- At exam level, more tokens generally means more processing and potentially higher cost/latency.

4. Retrieval Augmented Generation (RAG)

Core definition

RAG retrieves relevant information from an external data source and provides that information as context to a foundation model when generating a response. The model does not need to be retrained just to use the retrieved knowledge.

User question

Convert/query for retrieval

Retrieve relevant company/private/current information

Add retrieved information to model context

Foundation model generates grounded response

Why businesses use RAG

- Use private or proprietary company knowledge.
- Use information that changes frequently without repeatedly retraining the model.
- Ground responses in retrieved sources and reduce unsupported answers.
- Build enterprise question-answering over documents and knowledge sources.

Amazon Bedrock Knowledge Bases

Memory phrase

Amazon Bedrock Knowledge Bases = RAG + company/private knowledge + grounded responses.

- AWS documentation describes Knowledge Bases as a managed capability for implementing a RAG workflow from ingestion/retrieval through prompt augmentation.
- At exam level, if a scenario asks to connect a foundation model to proprietary documents without retraining, think RAG / Amazon Bedrock Knowledge Bases.

5. Embeddings and Vector Databases in RAG

Study explanation: embeddings are numerical vector representations used to compare semantic similarity.

A common RAG flow is:

Documents

Chunking

Embedding model converts chunks to vectors

Store/index vectors in a vector database
Convert user query to a vector
Similarity search retrieves relevant chunks
FM generates answer from retrieved context

Vector storage vs vector database

"Vector storage" simply means storing vectors. A vector database/vector-capable data store also indexes and searches vectors by similarity. In practice, OpenSearch, Aurora PostgreSQL + pgvector, RDS for PostgreSQL + pgvector, and Neptune Analytics can support vector search use cases in different ways.

6. AWS Services to Memorize for Task 3.1

Service	Primary identity	Task 3.1 memory phrase
Amazon Bedrock Knowledge Bases	Managed RAG capability	RAG -> company/private knowledge -> grounded answer
Amazon OpenSearch Service	Search and analytics service	search engine -> semantic/vector search -> embeddings
Amazon Aurora	AWS managed relational SQL database engine	relational SQL DB -> PostgreSQL/MySQL compatible -> can support vector use cases
Amazon Neptune	Graph database / graph analytics family	graph -> relationships/connected data -> vector capabilities in Neptune Analytics
Amazon RDS for PostgreSQL	Managed PostgreSQL relational database	managed PostgreSQL -> SQL relational DB -> pgvector/vector embeddings

6.1 OpenSearch vs Aurora vs RDS for PostgreSQL

Service	Do NOT memorize as...	Memorize as...
OpenSearch	"only stores/searches vectors"	A search engine/search service that can perform semantic and vector search.
Aurora	"NoSQL"	A relational SQL database engine built by AWS, compatible with PostgreSQL/MySQL.
RDS for PostgreSQL	"the same product as Aurora"	PostgreSQL itself, managed for you by Amazon RDS.

Aurora vs RDS PostgreSQL

RDS for PostgreSQL = AWS manages the PostgreSQL database engine. Aurora PostgreSQL = AWS-built Aurora relational database engine that is PostgreSQL-compatible. Both are relational/SQL; Aurora is not NoSQL.

7. FM Customization Approaches

The exam guide explicitly asks for cost tradeoffs among pre-training, fine-tuning, in-context learning, RAG, and model distillation. Use the following mental model:

Approach	What happens	Model weights change?	Fast exam keyword
Pre-training	Train a foundation model from a very large dataset to build general capabilities.	Yes	train from scratch / base FM
Fine-tuning	Continue training an existing pre-trained model on smaller, task/domain-specific examples.	Yes	specialize behavior/domain

In-context learning	Provide instructions/examples inside the current prompt/context.	No	examples in prompt
RAG	Retrieve external/current/private knowledge and place it in context at inference time.	No	external knowledge without retraining
Model distillation	Train a smaller student model to reproduce useful behavior/capabilities of a larger teacher model.	Student model is trained	large teacher -> smaller/cheaper student

7.1 The easiest analogy

Technique	Student analogy
Pre-training	The student goes through a huge general education from the beginning.
Fine-tuning	The already-educated student takes a specialized course.
In-context learning	The student is given a few worked examples right before answering this question.
RAG	The student is allowed to open a relevant reference book while answering.
Distillation	A very capable senior/teacher helps train a smaller junior/student to imitate useful capabilities.

Important distinction

Fine-tuning changes model weights. In-context learning and RAG do not. In-context learning changes only the current prompt/context; RAG adds retrieved external knowledge to the context.

8. Relative Cost Tradeoff - Exam-Level View

Approach	Typical cost/effort intuition	Why
Pre-training	Highest	Requires very large datasets and substantial compute to build a base model.
Fine-tuning	Higher than prompt-only approaches	Requires additional training of an existing model.
In-context learning	Low setup effort, but prompt tokens can add inference cost	No training; examples/instructions are sent in context.
RAG	Adds retrieval/index/storage infrastructure	Avoids retraining for frequently changing knowledge, but needs retrieval components.
Distillation	Training effort upfront, potentially lower inference cost later	Creates a smaller model intended to retain useful behavior while being more efficient.

Do not memorize exact prices

The exam objective asks for tradeoffs, not a universal dollar ranking. Actual cost depends on model, provider, data volume, tokens, infrastructure, and workload.

9. AI Agents

Core idea

A foundation model can generate an answer. An AI agent can use the model to reason about a goal, select tools or data sources, take actions, observe results, and continue a multi-step task.

User gives a goal

Agent plans/decides next step

Agent uses tools/APIs/knowledge/data

Agent observes result

Agent continues until task is completed

RAG vs Agent

RAG	Agent
Primary purpose: retrieve relevant knowledge and ground generation.	Primary purpose: complete a goal through actions/tools and potentially multiple steps.
Think: "get the right information."	Think: "do the task."
Can be used by an agent as one of its tools.	Can call RAG, APIs, databases, or other tools.

10. One-Page Keyword Cheat Sheet

Question clue	Think...
Creative / diverse response	Higher temperature
Predictable / consistent response	Lower temperature
Company/private/current knowledge	RAG
Managed RAG on AWS	Amazon Bedrock Knowledge Bases
Semantic search / vector search	Amazon OpenSearch Service
AWS relational database, PostgreSQL/MySQL compatible	Amazon Aurora
Graph / relationships / connected data	Amazon Neptune
Managed PostgreSQL	Amazon RDS for PostgreSQL
Train from scratch	Pre-training
Change/specialize model behavior or domain	Fine-tuning
Examples inside prompt, no weight change	In-context learning
External knowledge, no weight change	RAG
Large teacher -> smaller student	Model distillation
Image + text	Multimodal model
Many languages	Multilingual capability
Real-time interactive application	Low latency
Repeated shared prompt/context	Prompt caching
AI uses tools and takes actions	Agent

11. Final Review Checklist

- [] I can explain why low vs high temperature changes response behavior.
- [] I can explain RAG without saying it retrains the model.
- [] I can draw the flow: documents -> chunking -> embeddings -> vector database -> retrieval -> FM.
- [] I remember: Bedrock Knowledge Bases = managed RAG capability.
- [] I can distinguish OpenSearch, Aurora, Neptune, and RDS for PostgreSQL by their primary identity.
- [] I know Aurora and RDS for PostgreSQL are both relational/SQL; Aurora is not NoSQL.

- [] I can distinguish pre-training, fine-tuning, in-context learning, RAG, and distillation.
- [] I remember: in-context learning and RAG do not change the foundation model weights.
- [] I can distinguish RAG (retrieve knowledge) from an agent (use tools/take actions).

12. Sources and Scope Notes

Primary exam basis: AWS Certified AI Practitioner Exam Guide (AIF-C01), version 1.1, Task Statement 3.1.
The guide defines the required objectives but does not provide implementation-level detail for every service.

Supplementary AWS documentation used for study explanations:

- Amazon Bedrock User Guide - Knowledge Bases and retrieval/RAG workflow.
- Amazon OpenSearch Service Developer Guide - vector search and semantic search.
- Amazon Aurora documentation - Aurora as a fully managed relational database compatible with PostgreSQL/MySQL; Aurora PostgreSQL vector integration.
- AWS Prescriptive Guidance - vector database options for RAG, including RDS for PostgreSQL with pgvector and Aurora PostgreSQL.
- Neptune Analytics User Guide - graph analytics and vector similarity search.

Scope discipline

Where the exam guide only names a service as an example, this document keeps the exam-level takeaway simple. Technical details are included only to clarify distinctions that commonly cause confusion.